

## CCH は、書かれている仕様どおりに動いているか

ガードレール 15 ルール・文脈 13 項目・フック 20 イベント・配布バイナリを、実際にペイロードを流し込んで観測した結果です。コードを読んだだけの推測は含みません。

15 ルール中 13 が実測で正しく動いていた。壊れているのはルールではなく、**ルールへ情報を届ける配線**で、供給元が存在しない項目が 2 つある。加えて、他プロジェクトは 1 日古いバイナリで動いており、修正はリリースするまで届かない。

作成 2026-08-11    対象 v5.6.0 / branch fix/r04-agent-state-exception    測定 稼働バイナリへの直接投入  
位置づけ 派生物。正本ではない

### やさしく言うと

CCH は、AI が何か実行する直前に割り込んで「許可・確認・拒否」を決める仕組みです。中身は 2 段構えで、① **状況を集める**（誰が、どのモードで、どのプロジェクトで）→ ② **15 個のルールで判定する**、という順に動きます。

今回 15 ルール全部に実際の操作を投げ込んだところ、13 個は期待どおりに反応しました。反応しなかった 2 個（R07 と R08）は、ルール自体は正しく書かれているのに、  
①が**集めるはずの情報**を誰も渡していないため、永久に条件を満たしません。

もう 1 つ実務に効く事実があります。このリポジトリは 8/10 のバイナリで動いていますが、**他のプロジェクトは 8/9 のバイナリ**で動いていました。先日入れた修正はそちらに届いていません。

### breezing はもう止まらない？

止まりません。ただし支えているのは `~/.claude/settings.json` の `HARNESS_WORK_MODE=1` という 1 行だけです。外すと元に戻ります。

### 危険な操作は止まる？

止まります。実測で `sudo` ・ `--force` `push` ・ `--no-verify` ・ 秘密ファイルの `staging` ・ 保護ブランチへの `reset --hard` はすべて拒否されました。

### 今すぐ何かする必要は？

ありません。危険な操作の防御は生きています。ただしリリースしないと、他プロジェクトへ修正が届きません。

### いちばん怖いのはどれ？

R08 です。brezing のレビュー担当が実装に手を出せる状態で、機械的な歯止めが効いていません。

## 01 — STRUCTURE

# CCHは何でできているか

まず全体の骨格です。検証はこの各層に対して行いました。

| 層      | 実体                                                      | 今回の検証         |
|--------|---------------------------------------------------------|---------------|
| ガードレール | 15 ルール (R01～R15) + runtime floor 2 種。実行前に許可・確認・拒否を返す    | 全件に実操作を投入     |
| 文脈     | <code>BuildContext</code> が集める 13 項目。ルール の判定材料になる       | 全項目の供給元を追跡    |
| フック    | 20 イベント (PreToolUse / PostToolUse / SessionStart ほか)    | 登録状況と配布コピーの一致 |
| スキル    | 21 個 (plan / work / review / release / brezing ほか)      | ミラー同期を確認      |
| エージェント | 4 個 (worker / reviewer / advisor / test-wiring-auditor) | 存在確認          |
| CLI    | 36 サブコマンド。Go 実装、内部パッケージ 48                              | ゲート実行         |

## 判定はこの順に流れる

### ① AI がツールを呼ぼうとする

Write / Edit / Bash / Read など



### ② 状況を集める (BuildContext)

13 項目。ここが今回の弱点



### ③ 15 ルールを上から順に評価する

最初に一致したルールの判定で確定する



### ④ 許可 / 確認 / 拒否 を返す

確認＝作業が止まる。拒否＝その場で終わる

## 02 — MEASURED

# 15 ルールの実測結果

1 ルールずつ実際のペイロードをバイナリに流し込み、返ってきた判定を記録しました。行を選ぶと、投入した操作と生の応答が右に出ます。

|      |                   |       |
|------|-------------------|-------|
| R01  | sudo の禁止          | 拒否    |
| R02  | 保護パスへの書き込み        | 許可+警告 |
| R03  | シェル経由の保護パス書き込み    | 許可+警告 |
| R04  | プロジェクト外への書き込み     | 確認    |
| R04b | R04 — 自分の記憶ディレクトリ | 無言で許可 |
| R05  | 破壊的な削除            | 拒否    |
| R06  | 強制 push の禁止       | 拒否    |
|      |                   |       |

|       |                         |       |
|-------|-------------------------|-------|
| R07   | codex モードでの直接書き込み禁止     | 無反応   |
| R08   | レビュー担当の書き込み禁止           | 無反応   |
| R09   | 秘密ファイル読み取りの警告           | 許可+警告 |
| R10   | git の検証バイパス禁止           | 拒否    |
| R11   | 保護ブランチの reset --hard    | 拒否    |
| R11b  | R11 — HEAD~1 を指した場合     | 無言で許可 |
| R12   | main への直接 push          | 確認    |
| R13   | 重要ファイル変更の警告             | 許可+警告 |
| R14   | src 変更にテストを要求           | 無言で許可 |
| R15   | 秘密ファイルの staging 禁止      | 拒否    |
| FLOOR | runtime floor — 秘密の読み取り | 拒否    |

### 03 — WIRING

## 文脈 13 項目に、値を入れる仕組みはあるか

ルールに情報を届ける側です。ここが切れていると、ルールは正しく書かれていても永久に発火しません。

| 文脈項目         | 供給元                                                          | 状態   |
|--------------|--------------------------------------------------------------|------|
| Input        | フックのペイロード                                                    | 実配線  |
| ProjectRoot  | 自動解決                                                         | 実配線  |
| WorkMode     | env (operator が手で設定) / SQLite<br>( work-mode CLI は識別子不一致で無効) | 応急処置 |
| CodexMode    | 供給元なし → R07 が起動不能                                            | 配線なし |
| BreezingRole | 供給元なし → R08 が起動不能                                            | 配線なし |

|                                  |                                                 |     |
|----------------------------------|-------------------------------------------------|-----|
| ProtectedBranchPushPolicy        | harness.toml の protectedBranchPush = "ask"      | 実配線 |
| ConsumePlanPreapproval           | .claude/state/plan-preapprovals.json (5.2KB 実在) | 実配線 |
| ProtectedPathAskList             | harness.toml (任意機能。今は未使用)                       | 実配線 |
| TddEnforceLevel / TddHookEnabled | harness.toml の [tdd.enforce] level = "off"      | 実配線 |
| TddBypass ほか 3 項目                | env (手動の一時解除スイッチ。設計どおり operator 供給)             | 実配線 |

go/internal/breezing の中身も確認しましたが、role という概念そのものはありません。R08 は「実装されているが、起動する手段が存在しない」で確定です。

#### 04 — PROVENANCE

## どのバイナリが実際に動いているか

今回いちばん実務に効く発見です。フックの起動スクリプトは bin/harness を持つ場所を上から順に探し、最初に見つかったものを使います。

## 探索順と、実際に一致する場所

### ① CLAUDE\_PLUGIN\_ROOT

未設定



### ② CLAUDE\_PROJECT\_DIR / 現在地

このリポジトリはここで一致 → 8/10 18:12 のバイナリ (R04 修正あり)



### ③ marketplaces クローン

他プロジェクトはここに落ちる → 8/9 16:19 のバイナリ (R04 修正なし)



### ④ plugins/cache/.../5.6.0

bin/ ディレクトリが存在しない (実測)

| 場所               | バイナリ | 日付         | 記憶ディレクトリへの書き込み             |
|------------------|------|------------|----------------------------|
| このリポジトリ          | あり   | 8/10 18:12 | 無言で許可<br>(修正が効いている)        |
| marketplace クローン | あり   | 8/9 16:19  | <span>ask</span> 修正が入っていない |
| cache 5.6.0      | なし   | —          | —                          |

他プロジェクトで確認ダイアログが出ない唯一の理由は、環境変数

**HARNESS\_WORK\_MODE=1** です。これがリリースの意味を正確に定義します。

リリースするまで、コードの修正は他プロジェクトに一切届きません。

## 05 — SIDE EFFECT

## 応急処置には副作用があった

**HARNESS\_WORK\_MODE=1** が Bash の子プロセスにも継承されていることを、このセッションで実測しました。その結果、テストプロセスにも漏れています。

## 同じテストを、環境変数の有無だけ変えて実行した

```
go test ./...          (今の環境そのまま) → 22 ok / 1 FAIL
FAIL: TestEvaluatePreTool_AuditRootDoesNotChangeRuleEvaluationRoot
```

```
同じテスト、HARNESS_WORK_MODE を外す → ok
```

テスト側が環境変数を掃除していないため、応急処置がガードレールのテストを 1 件落としています。 `t.Setenv` で空にする 1 行で直ります。

| ゲート                                   | 結果                    | 備考                        |
|---------------------------------------|-----------------------|---------------------------|
| <code>go test ./...</code>            | 22 ok / <b>1 FAIL</b> | 上記の環境変数漏れが原因。外せば緑         |
| <code>tests/validate-plugin.sh</code> | 136 / 0 / 0           | 2 回実行して一致                 |
| skill ミラー                             | in-sync               | codex / opencode とも       |
| フック配線                                 | 20 イベント               | tracked と plugin コピーが完全一致 |

06 — DRIFT

## 仕様と実装のズレ

| # | 内容                                                                                                                       | 深刻度 |
|---|--------------------------------------------------------------------------------------------------------------------------|-----|
| 1 | R07 / R08 が起動不能。codex モードの直接書き込み禁止と、レビュー担当の書き込み禁止が効いていない                                                                 | 中   |
| 2 | <code>work-mode</code> CLI が識別子不一致で無効 (Plans.md 132.7 として起票済み)                                                           | 中   |
| 3 | 応急処置の環境変数がテストを 1 件落とす                                                                                                    | 中   |
| 4 | 他プロジェクトは 8/9 バイナリ。R04 の修正が未達                                                                                             | 中   |
| 5 | <code>spec.md:34</code> と <code>CLAUDE.md</code> が「R01-R13」表記。実体は R15 までである                                              | 低   |
| 6 | <code>CLAUDE.md</code> が <code>settings / workflows</code> を「多層防御」と書くが、ガードレール層は警告のみ。deny は <code>permissions</code> 層の単層 | 低   |
| 7 | Phase 132.4 で入れたゲートが <code>env</code> キーだけを見るため誤検知 6/8。設定ファイル経由の供給を見ていない                                                 | 低   |

## 完成図

現在と完成形を、同じ 5 項目の骨格で並べます。

| 項目       | 今                                                      | 完成形                          |
|----------|--------------------------------------------------------|------------------------------|
| ルール      | 15 個中 13 稼働 / 2 起動不能                                   | 15 個すべてに「立てる手段」が実在           |
| 文脈       | 13 項目中 11 実配線 / 1 応急処置 / 2 供給元なし                       | 13 項目すべてに供給元。CI が毎回確認        |
| breezing | 環境変数 1 行で停止回避。<br>ただしテストを 1 件壊す                        | run 開始時に自動で切り替わる。<br>環境変数は不要 |
| 他プロジェクト  | 8/9 バイナリ。修正が未達                                         | リリース済みバイナリで同じ挙動              |
| 検査       | ルール⇔テストの対応はゲート化済<br>( rulecoverage )。ルール⇔<br>文脈の配線は無検査 | 両方の軸を CI が確認                 |

`go/internal/rulecoverage` を開いて確認しました。

これは**すでに存在する検査ゲート**で、ルール ID をテスト・deny baseline・shell ゲートと突き合わせます。ただし見ている軸が「ルールにテストがあるか」なので、テストを持つ R07 / R08 を「カバー済み」と判定してしまいます。

検査の仕組みと置き場所は既にあり、足りないのは軸が 1 本だけです。

## 次の一手（推奨順）

### 1 文脈 13 項目の配線テストを `rulecoverage` に追加する

なぜ: 既存ゲートに軸を 1 本足すだけ。R07 / R08 / 132.3 の失敗が同時に赤くなる。  
製品コードを触らないので後戻りリスクが最小。

### 2 テストの環境変数を掃除する ( `t.Setenv` で空に )

なぜ: 今 FAIL している 1 件が直る。1 行の変更。

### 3 `work-mode` の識別子を修正する (132.7)

なぜ: 直れば環境変数の応急処置を外せる。手順 1 が先にないと、直ったことを確認できない。

### 4 R07 / R08 に供給元を実装する (132.6)

なぜ: R08 はレビュー担当の独立性を守る歯止め。今はそれが機械的に効いていない。

なぜ: 他プロジェクトへ R04 の修正を届ける唯一の手段。外部公開にあたるため、明示的な判断が要る。

## 判定

危険な操作に対する防御は実測で生きている。 `sudo` ・履歴破壊・秘密ファイルの staging ・保護ブランチ操作は、すべて拒否された。

**今すぐ手当てが必要な故障はない。**

一方で、構造上の弱点が 1 つある。ルールを検査する仕組みはあるが、

**ルールに情報を届ける配線を検査する仕組みがない。** R07 / R08

が長期間死んでいたことも、work-mode の配線が効いていなかったことも、同じ 1 つの穴から出ている。手順 1 と 2 はテストだけで塞げる。

## 正直な但し書き

- 前回「ルールは 13 個」と報告したのは R01~R13 を数えたもので、実エンジンは 15 個 (R14 / R15 が後から追加されている)。本レポートは 15 で統一している。
- R05 の確認 (ask) は今回一度も観測できなかった。runtime floor が先に拒否するため。「R05 = 確認が出る」と書くと実測と食い違う。
- `git reset --hard HEAD~1` はガードレールを素通りする。R11 は保護ブランチ名を指した時だけ発火する設計。止めているのは `~/.claude/settings.json` の deny (実在を確認済み) で、層は 1 枚。
- ルールの網羅性そのものは検証していない。「登録されている 15 個が動くか」を測ったのであって、「守るべき操作がすべて 15 個に含まれているか」は別の問い。
- Codex / Cursor ホストでの実挙動は今回測っていない。測ったのは Claude ホストの経路のみ。
- 本レポートは実装を伴わない。記載した修正はいずれも未着手。

## 出典 (SSOT)

- `go/internal/policy/rules.go` — 15 ルールの定義 (R01:133 / R04:209 / R07:288 / R08:303 / R11:375 / R13:431)
- `go/internal/guardrail/pre_tool.go:110` — `BuildContext` (文脈 13 項目の組み立て)

- `go/pkg/hookproto` — `RuleContext` 構造体 (13 フィールド)
- `go/internal/rulecoverage/coverage.go` — 既存のルール⇔検査ゲート
- `harness.toml` — `protectedBranchPush` (24) / `[tdd.enforce] level` (152)
- `~/.claude/settings.json` — `permissions deny` 38 件 / `env / sandbox` 無効
- `Plans.md` Phase 132.6・132.7 — 未着手タスクの起票内容
- `.claude/rules/defense-layer-blast-radius.md` — 防御層ごとの影響範囲
- `hooks/hooks.json` / `.claude-plugin/hooks.json` — 20 イベントの配線

## 測定方法と制約

- 各ルールに対し、稼働バイナリ `bin/harness hook pre-tool --host claude` へ標準入力で実ペイロードを投入し、返却 JSON を記録した。
- 環境変数の影響を分離するため、`HARNESS_WORK_MODE` を「外した状態」と「1の状態」の2通りで全件を測定した。
- バイナリの出どころは、フック起動スクリプトの探索順に沿って各候補の `bin/harness` の実在と更新日時を確認し、marketplace クローンにも同一ペイロードを投入して差を観測した。
- 制約: 測定はすべて macOS / Claude ホスト / このリポジトリを起点に行った。Codex・Cursor ホスト、および他プロジェクトを起点とした実セッションでの挙動は測定していない。
- 制約: ルールの網羅性（守るべき操作が漏れなく 15 個に入っているか）は検証範囲外。

本レポートは 2026-08-11 時点の測定にもとづく派生物であり、仕様の正本ではない。正本は上記 SSOT を参照すること。